SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA1711**

COURSE OUTCOME COVERED: ***CO2:*** *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 50%

QUESTIONS: 5 **Duration: 1 Hour**

Total Marks:50

Annexure A

Scenario Based Assignment

Code of Conduct:

I Arulmozhi Chozhan - 192512344(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Scenario 1: AI Drone Delivery in a Flood-Affected Region

Task 1: Behaviour of Greedy Best-First Search

Greedy Best-First Search (GBFS) expands the node that appears closest to the goal, using only the heuristic function h(n) — the straight-line distance to the drop-off point — and ignoring the actual path cost g(n) already incurred. In this scenario the drone would always fly toward whichever waypoint looks geometrically nearest to the destination, without weighing terrain difficulty or accumulated risk.

Strengths

- Very fast: because it only follows h(n), it explores far fewer nodes than cost-based methods and returns a route quickly — valuable when medicines must be dispatched with minimum delay.
- Low computational and memory overhead, suitable for a drone with limited onboard processing.

Weaknesses under uncertainty

- Not optimal: it can choose a route that looks short on the map but crosses a newly flooded or blocked road, since it never reconsiders real movement cost.
- Not complete in general graphs: it can get trapped oscillating between nodes that look attractive but lead into a dead end (e.g., a flooded cul-de-sac), especially with only partial real-time updates.
- Highly sensitive to heuristic quality — a poor or now-inaccurate straight-line estimate (e.g., a bridge that no longer exists) misleads the whole search.

Overall, GBFS trades safety and optimality for raw speed, which is risky when blocked paths and variable terrain cost can silently invalidate the 'shortest-looking' route.

Task 2: Behaviour of A* Search

A* evaluates every node with f(n) = g(n) + h(n), combining the real cost already travelled (g) with the estimated remaining cost (h). It therefore does not just chase the node that looks closest — it balances 'how far have I already paid to get here' against 'how far do I still expect to go.' In the drone scenario, g(n) would accumulate the true weather-adjusted, terrain-adjusted travel cost, while h(n) remains the straight-line distance.

As long as the heuristic is admissible (never overestimates true remaining cost) and consistent, A* is guaranteed to return the optimal (minimum-cost) route. It naturally re-ranks paths as real costs change: a route that initially looked cheap but crosses expensive/blocked terrain will be pushed down the frontier once its true g(n) is known, while GBFS would keep favouring it purely on geometric proximity.

Task 3: Impact of Heuristic Accuracy on Both Algorithms

- Greedy Best-First Search: is almost entirely governed by h(n). An accurate heuristic makes it behave close to optimal by luck; an inaccurate or stale heuristic (e.g., a road that has just flooded) can send the drone down a dangerous or dead-end path, since g(n) is never used to correct the mistake.
- A*: heuristic accuracy affects efficiency, not correctness. A closer-to-true heuristic prunes the search space faster and finds the optimal route with fewer node expansions. An under-informed heuristic (e.g., h = 0) degrades A* toward Dijkstra's algorithm — still optimal, but slower — whereas an inadmissible (overestimating) heuristic can break A*'s optimality guarantee.

In short: for GBFS, heuristic accuracy determines correctness itself; for A*, it determines speed while correctness is preserved as long as the heuristic stays admissible.

Task 4: Effect of Dynamic Changes (Blocked Paths)

- Both algorithms were designed for essentially static graphs; a newly blocked road invalidates parts of the search tree already expanded.
- Greedy Best-First Search reacts poorly: since it never tracked real cost, it has no mechanism to detect that a 'promising' node is now unreachable until it actually tries to move through it, often causing late replanning and wasted time.
- A* can be adapted more gracefully using incremental/real-time variants (e.g., D* Lite, Lifelong Planning A*, or simply re-running A* from the drone's current position) because its g(n) bookkeeping makes it straightforward to update costs and re-optimize only the affected part of the path.
- In general, dynamic environments increase re-planning frequency and computational load for both, and favour algorithms that support incremental replanning over one-shot search.

Task 5: Recommendation and Justification

A* (or a real-time/incremental variant such as D* Lite) is the most suitable choice for the drone delivery scenario. It guarantees an optimal, minimum-risk route whenever the heuristic is admissible, correctly incorporates variable terrain and weather cost through g(n), and can be re-invoked efficiently as partial real-time updates about blocked paths arrive. Greedy Best-First Search may be used only as a fast fallback when the drone must act instantly and a near-optimal, low-latency answer is acceptable, but given that safety and correctness of the delivery route matter as much as speed, A*'s balance of optimality and adaptability makes it the safer real-time decision-maker.

Scenario 2: Smart City Traffic Signal Optimization

Task 1: Formulation as an Optimization Problem

This is a local-search / optimization problem rather than a path-finding problem, because there is no single start-to-goal path to discover — instead, a complete assignment of signal timings (a state) must be found that minimizes a cost/objective function combining total waiting time, fuel consumption, and congestion. The state space consists of every possible combination of green/red durations across all intersections, which grows combinatorially with the number of intersections, so it cannot be enumerated exhaustively.

Traditional systematic (tree) search — such as BFS, DFS, or even A* — is inefficient here because: (1) the state space is astronomically large and has no natural goal test, only a quality score to optimize; (2) the environment is continuously changing, so a path computed now may already be stale by the time it is applied; and (3) these methods keep and explore many alternative branches simultaneously, which is far more memory- and time-expensive than simply moving through the space of complete solutions and improving them, which is what local search does.

Task 2: Hill Climbing and Its Limitations

Hill Climbing starts with one candidate signal-timing configuration and repeatedly moves to a neighbouring configuration (e.g., adjusting one intersection's green time slightly) as long as it improves the objective (lower waiting time/fuel/congestion). It keeps no search tree and no backtracking — only the current best state is retained.

Limitations

- Local maxima: it can settle on a configuration that is better than all its immediate neighbours but is not the best possible configuration city-wide.

- Plateaus: large regions where neighbouring configurations score equally, giving no gradient to climb, so the algorithm wanders or stalls.
- Ridges: sequences of moves that individually look non-improving even though a combined move would improve the score, causing the simple algorithm to stop prematurely.
- No mechanism to escape a poor region once trapped, since it never accepts a worse intermediate state.

Task 3: Local Maxima, Plateaus, and Ridges — Real-World Interpretation

- Local maxima: a set of signal timings that improves flow at nearby junctions but locks in a citywide pattern where an obvious better balance (e.g., synchronised 'green wave' timing along an arterial road) is never reached, because reaching it needs several junctions to change together.
- Plateaus: during low-traffic periods (e.g., late night), many different signal timings produce almost identical waiting times, so the search has no clear signal about which direction to adjust next.
- Ridges: improving one intersection's timing while worsening another's may look like a step backward in isolation, even though jointly retiming both (e.g., coordinating a whole corridor) reduces total city-wide congestion — the ridge in the search landscape hides this benefit from single-step search.

Task 4: Overcoming These Limitations — Simulated Annealing / Genetic Algorithms

Simulated Annealing (SA) introduces controlled randomness: it sometimes accepts a worse configuration with a probability that decreases over time ('cooling schedule'), allowing the search to escape local maxima and cross plateaus early on, while converging to a strong solution as the acceptance of worse moves is gradually reduced.

Genetic Algorithms (GA) maintain a population of candidate signal-timing solutions and evolve them through selection, crossover, and mutation across generations. Because many diverse solutions are explored in parallel, GAs are naturally resistant to any single local maximum, can jump over plateaus through mutation, and can combine (crossover) good partial solutions from different intersections — directly addressing the ridge problem where a joint change is needed for improvement.

Both approaches trade a guarantee of optimality for a much higher chance of finding a near-optimal, robust solution in a huge, dynamic, non-convex search space.

Task 5: Recommended Approach and Justification

A Genetic Algorithm, optionally combined with elements of Simulated Annealing for local refinement (a hybrid/memetic approach), is the most suitable recommendation. City-wide signal optimization needs solutions that scale to hundreds of intersections (GA's population-based, parallelisable search scales well), adapt to continuously changing traffic patterns (recomputing/re-evolving populations incrementally), and avoid being trapped by the many local optima inherent in coordinating multiple correlated intersections. Pure Hill Climbing is too easily trapped, while GA/SA offer the scalability and adaptability the real-time, dynamic smart-city environment requires.

Scenario 3: Autonomous Mars Rover Exploration

Task 1: Why an Online Search Agent Instead of Offline Search

Offline (classical) search assumes the agent has complete knowledge of the environment before it starts moving — it computes an entire solution path first, then executes it blindly. The Mars rover, however, does not have a complete map of the unknown terrain in advance; craters, rocks, and hazards are only

discovered as it senses its surroundings. An online search agent interleaves computation and action: it takes one action, observes the actual outcome, updates its knowledge, and then decides the next action. This is essential here because committing to a full offline plan would be unsafe — the plan could lead straight into an undetected hazard that only becomes visible once the rover is close enough to sense it.

Task 2: Challenges of Partial Observability and Dynamic Environments

- Partial observability: the rover's sensors reveal only a local neighbourhood at any time, so it cannot evaluate the true cost or safety of distant paths in advance and may need to backtrack after discovering a hazard.
- Risk of irreversible actions: some moves (e.g., driving into a crater) cannot be undone, so the rover must search cautiously, sometimes even preferring safer, sub-optimal moves over risky shortcuts.
- Dynamic terrain/uncertainty: dust storms, shifting sand, or newly detected rocks can change the environment between planning cycles, invalidating previously gathered knowledge.
- High cost of exploration: each movement consumes limited power/time, so the rover must balance thorough exploration against efficient progress toward its sample-collection goal.

Task 3: How the Rover Builds and Updates Its Internal Model

As the rover moves, it uses onboard sensors (cameras, LIDAR, etc.) to perceive the immediately visible terrain and merges these observations into an internal map/graph of explored versus unexplored regions. Each time it takes an action and senses the result, it updates edge costs (e.g., marking a cell as a hazard or as traversable) and revises its belief state. Algorithms such as Learning Real-Time A* (LRTA*) exemplify this: the rover maintains a current cost estimate for each visited state, updates that estimate whenever the observed cost differs from the expectation, and uses the improved estimates to make better decisions on future encounters with the same or similar terrain — effectively learning a more accurate heuristic through experience.

Task 4: Online Search vs Uninformed and Informed (Offline) Search

- Uninformed offline search (e.g., BFS, DFS, uniform-cost search) requires the full state space to be known upfront and explores blindly without any heuristic guidance — completely impractical for unmapped Martian terrain.
- Informed offline search (e.g., A*) improves efficiency with a heuristic but still assumes the whole map/cost structure is known before execution begins, which the rover does not have.
- Online search does not require prior knowledge of the full map; it acts, observes, and learns concurrently, at the cost of potentially non-optimal, sometimes more expensive paths (since mistakes can only be discovered after being made) and the risk of thrashing between unexplored options.

In short, offline methods trade real-world applicability for optimality guarantees that only hold given full prior knowledge, while online search trades some optimality for the ability to function safely under partial observability.

Task 5: Justification of the Most Suitable Approach

An online, informed search agent — such as LRTA* — is the most suitable approach. It combines the safety benefits of interleaving action and observation (necessary because the rover cannot know the full terrain in advance and some hazards are irreversible) with the efficiency benefits of heuristic guidance (using estimated distance/cost to the goal to bias exploration sensibly rather than randomly). Its ability to learn and refine cost estimates over repeated encounters with similar terrain makes it well suited to a long-

duration mission where the rover progressively builds a more accurate internal model, directly addressing the mission's core requirements of uncertainty, adaptability, and safety.

Scenario 4: AI-Generated University Exam Timetable

Task 1: Formulation as a Constraint Satisfaction Problem (CSP)

A CSP is defined by a set of variables, their domains, and the constraints relating them. The exam timetabling problem fits naturally: each course/exam is a variable that must be assigned a value (a time slot, possibly paired with a hall), subject to constraints such as no student having two exams simultaneously, limited hall capacity, prerequisite subject ordering, faculty availability, and special accommodations. The goal is to find a complete assignment of time slots (and rooms) to all exams that satisfies every constraint — a textbook CSP rather than a path-search problem, since there is no notion of 'distance travelled,' only validity of the final assignment.

Task 2: Variables, Domains, and Constraints

- Variables: one variable per exam/course to be scheduled, e.g., $Exam_1$, $Exam_2$, … $Exam_n$.
- Domains: for each exam variable, the domain is the set of available (time slot, hall) pairs, e.g., {Day1-Slot1-HallA, Day1-Slot1-HallB, Day1-Slot2-HallA, …}, restricted to slots within the exam window and halls with sufficient capacity.
- Unary constraints: apply to a single variable, e.g., an exam requiring special accommodation must be assigned a hall equipped for it; a subject may be restricted to specific permitted slots.
- Binary constraints: relate pairs of variables, e.g., two exams taken by overlapping sets of students cannot share the same slot (student clash constraint); Subject A must be scheduled strictly before Subject B (precedence constraint).
- Global/resource constraints: hall-capacity and faculty-availability constraints link many variables at once, since multiple exams competing for the same hall or invigilator in the same slot must not overlap.

Task 3: How Backtracking Search Works in This Scenario

Backtracking search assigns exams to slots one at a time (e.g., in the order of most-constrained courses first), checking after each assignment whether all constraints involving already-assigned variables are still satisfied. If a partial assignment violates a constraint (say, assigning $Exam_3$ to the same slot as $Exam_1$ when they share students), the algorithm backtracks — undoes the most recent assignment(s) — and tries the next value in the domain. This depth-first process continues, incrementally building toward a full valid timetable, until either every exam is successfully scheduled or all possibilities are exhausted (indicating the constraints as given are infeasible).

Task 4: How Constraint Propagation (e.g., Forward Checking) Improves Efficiency

Plain backtracking only detects a conflict after making an assignment, which can waste significant effort exploring branches that were doomed from the start. Forward checking, applied whenever a variable is assigned, immediately removes now-invalid values from the domains of all as-yet-unassigned variables connected to it by a constraint (e.g., once $Exam_1$ is fixed to Day1-Slot1, forward checking removes Day1-Slot1 from the domain of every exam that shares a student with $Exam_1$). This lets the algorithm detect a dead end far earlier — if any domain becomes empty, backtracking is triggered immediately rather than after several more wasted assignments. More advanced propagation (e.g., arc-consistency/AC-3) can prune even further by enforcing consistency across all constraint arcs, substantially reducing the search tree explored for large timetabling instances.

Task 5: Real-World Challenges and Best Strategy for Scalability

- As the number of courses/students grows, the constraint graph becomes dense, and plain backtracking with forward checking alone can still be slow; variable- and value-ordering heuristics (e.g., Minimum Remaining Values to pick the most constrained exam next, and Least Constraining Value to try the option that eliminates the fewest choices for others) are important accelerators.
- Soft constraints (faculty preference, student convenience) versus hard constraints (no clashes) often need to be handled together, which pure CSP formulations must extend using weighted/optimization variants.
- Special accommodations and faculty availability add extra unary/global constraints that can fragment otherwise-good slots, requiring either problem-specific heuristics or relaxation strategies when no fully feasible assignment exists.

The best overall strategy is backtracking search combined with constraint propagation (forward checking or arc-consistency) and intelligent variable/value ordering heuristics; for very large institutions, this can further be hybridized with local-search repair (e.g., min-conflicts) to scale to hundreds of courses while remaining responsive to last-minute changes.

Scenario 5: Strategic Game AI (Minimax & Alpha-Beta Pruning)

Task 1: How the Minimax Algorithm Models This Problem

Minimax models the game as a tree in which alternating levels represent the AI's turn (the MAX player, trying to maximize the evaluated outcome) and the opponent's turn (the MIN player, assumed to play optimally to minimize the AI's outcome). Starting from the current game state, the algorithm recursively expands all possible move sequences down to some depth or terminal states, evaluates the resulting positions, and propagates values back up the tree: MAX nodes take the maximum of their children's values, MIN nodes take the minimum. The AI then chooses the move at the root leading to the branch with the best guaranteed value assuming a perfectly rational, worst-case opponent — exactly the adversarial situation of an AI competing against a human player.

Task 2: Computational Challenges of Large Game Trees

- Exponential growth: the number of nodes grows as b^d, where b is the branching factor (possible moves per turn) and d is the search depth — real-time strategy games with many units and actions have a very large b, making full-depth search intractable.
- Strict time limits: the AI must decide within a bounded time window, so it cannot always search to a terminal (game-over) state and must cut off search at a limited depth, relying on a heuristic evaluation function for non-terminal states.
- Memory cost: storing/expanding the full tree (or even a wide frontier of it) can exceed practical memory limits for complex game states.
- Evaluating intermediate states accurately is itself costly for complex games with many features (unit positions, resources, terrain, etc.).

Task 3: How Alpha-Beta Pruning Improves Efficiency

Alpha-Beta pruning augments Minimax with two bounds — α (the best value MAX can currently guarantee) and β (the best value MIN can currently guarantee) — passed down the tree during the search. Whenever a node's value would not change the final decision at the root (specifically, when a node's evaluated value falls outside the current $[\alpha, \beta]$ window — i.e., $\alpha \geq \beta$), the remaining branches under that

node are pruned without being explored, because no rational opponent would ever allow the game to reach a state that provably will not be selected.

This yields exactly the same decision as plain Minimax but explores far fewer nodes: in the best case (with optimal move ordering, e.g., trying the most promising moves first), the effective branching factor is reduced from b to roughly $\sqrt{b}$, meaning the algorithm can search roughly twice as deep in the same time budget. This directly addresses the real-time strategy game's need to search deep, complex trees within strict time limits.

Task 4: Designing an Evaluation Function

Since the search must be cut off before reaching terminal states, a heuristic evaluation function estimates how favourable a given (non-terminal) game state is for the AI. A reasonable evaluation function for a real-time strategy game would combine weighted factors such as:

- Material/unit balance: number and strength of the AI's units versus the opponent's.
- Resource control: quantity of resources gathered or territory/strategic points held.
- Positional advantage: unit positioning relative to choke points, high ground, or the opponent's base.
- Threat/vulnerability assessment: exposure of the AI's units versus the opponent's, and immediate tactical threats.
- Tempo/mobility: number of available future actions, reflecting flexibility of the position.

Each factor is normalized and combined via weighted summation, with the weights tuned (e.g., through self-play or machine learning) so that the function reliably reflects true winning chances, since Minimax/Alpha-Beta's quality is only as good as this evaluation function when full-depth search is infeasible.

Task 5: Strategy for Balancing Optimality and Real-Time Performance

- Iterative deepening: run Minimax with Alpha-Beta at increasing depth limits, always keeping the best result found so far, so the AI can be interrupted at any time and still return the best move found within the time limit.
- Move ordering: search the most promising moves first (e.g., using results from previous iterations or heuristics) to maximize the pruning benefit of Alpha-Beta.
- Transposition tables: cache previously evaluated states reached via different move orders to avoid redundant computation.
- Selective/quiescence search: extend search slightly in highly volatile positions (e.g., ongoing combat) to avoid misjudging a state that is about to change sharply, while pruning aggressively in stable positions.

Together, these techniques let the AI approximate optimal play — the guarantee Minimax provides in principle — while respecting the strict real-time constraints of the game, making Minimax with Alpha-Beta pruning, iterative deepening, and a well-tuned evaluation function the recommended overall strategy.

Note on Rubric Alignment

Each scenario answer above addresses the five rubric criteria from the original assignment: it demonstrates complete understanding of the scenario, identifies the key issues and constraints involved, applies the relevant AI search/CSP/game-theory concepts, makes and justifies a clear final recommendation or decision, and is presented in a clearly structured, task-by-task format for readability.